3. Tipos de Dados, Variáveis, Constantes e Strings

A maioria dos algoritmos que desenvolvemos poderá ser executada por computador, e neste caso temos os denominados **Algoritmos Computacionais**.

Já os **Algoritmos Não Computacionais** são aqueles cuja seqüência de passos, a princípio não pode ser executada por um computador, como por exemplo, as instruções contidas em um manual de instalação de um *DVD Player* em um automóvel.

Para os propósitos desta disciplina será considerada a construção de Algoritmos Computacionais.

Para tanto, os conceitos a seguir tem como fundamento a linguagem de programação C++, contida no ambiente de desenvolvimento integrado (IDE – *Integrated Development Environment*) denominado Dev-C++, que é constituído, entre outros, de um editor de textos e do compilador C++, para desenvolvimento de programas a partir dos algoritmos apresentados e exercitados.

3.1 Tipos de Dados - Conceitos

Computadores trabalham processando informações contidas em sua memória, principalmente na RAM (*Random Access Memory*), conforme ilustrados a seguir.

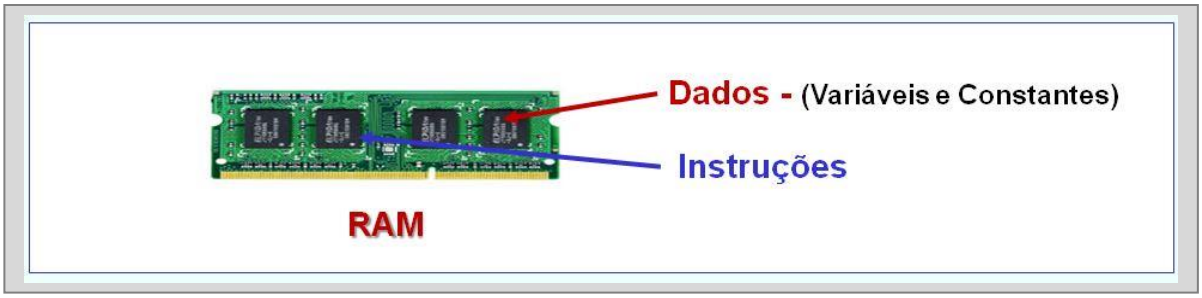
As informações são divididas, basicamente, em dois tipos:

- **Instruções**

  Comandam o funcionamento da máquina e determinam a maneira como devem ser tratados os dados.

- **Dados**

  Correspondem à porção das informações a serem processadas pelo computador.

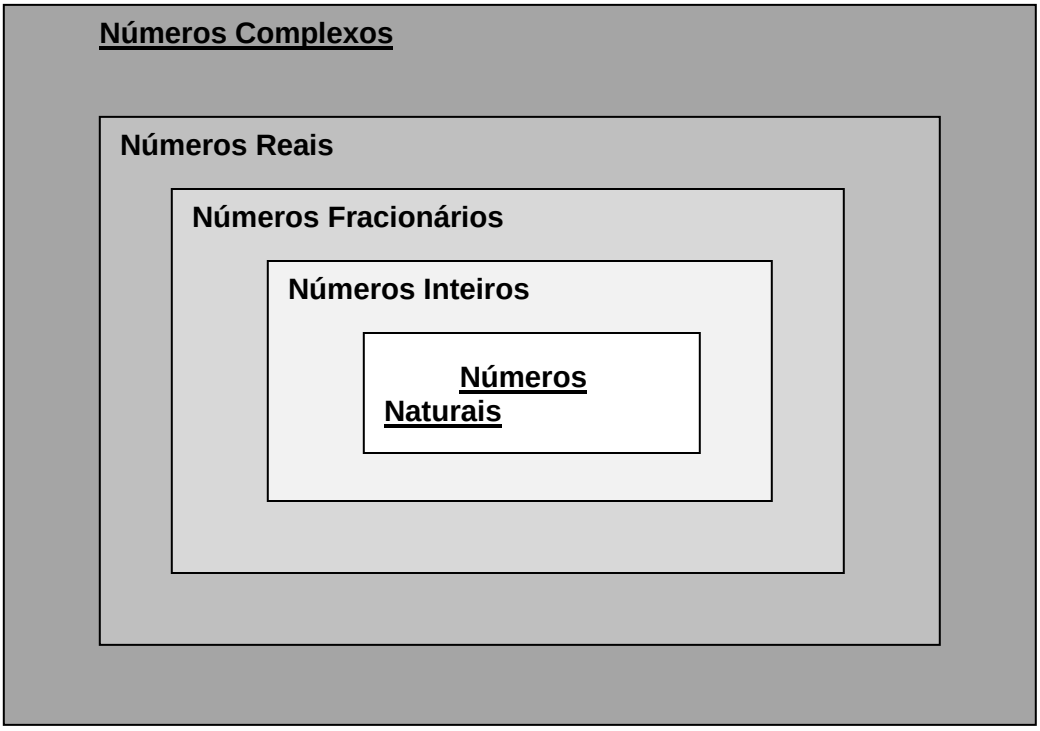


Nossa memória também processa dados levando-se em consideração os seus Tipos.

Dados podem ser Numéricos, Alfabéticos ou Alfanuméricos e Lógicos.

✓ **Dados Numéricos**: contém somente números.

Conforme teoria dos números e dos conjuntos, ilustrado no gráfico a seguir observa-se as divisões dos números em:



- ✓ **Dados Alfabéticos**: contém somente letras.
- ✓ **Dados Alfanuméricos**: pode conter Letras, números e caracteres especiais.
- ✓ **Dados Lógicos**: pode conter somente dois valores – **True** (verdadeiro) ou **False** (falso).

Tipos de dados que os computadores entendem e processam

Sem preocuparem-se com uma linguagem de programação específica, os computadores processam:

Dados Numéricos Inteiros

São constituídos por **Números Inteiros sem decimais ou frações**, e podem ser **positivos** ou **negativos**.

Exemplo: 0 35 -24 +128

Dados Numéricos Reais

São constituídos por **Números pertencentes ao conjunto de números reais**, possuem **decimais** ou **frações**, e podem ser **positivos** ou **negativos**.

Exemplo: 33.01 **0** -13.35 **156** -157.

Observação: embora a representação do número 0 e 156 possam ser confundidas com número inteiro, para afirmar que esses são números reais (com decimais), **deve-se analisar o contexto da sua utilização**.

Por exemplo, se 156 refere-se a metros de determinado material, então é um número real, que pode se entendido como 156.00. Da mesma forma o número 0, ou seja, se pertence ao contexto de representar uma quantidade em metros ou kilos, então, podemos entender como **0.0**.

Conforme já mencionado, os algoritmos desenvolvidos nesta disciplina serão codificados, compilados, testados e executados, em ambiente de programação C++, e para tanto esses algoritmos irão manipular os conjuntos de números **INTEIROS** e **REAIS** que são "entendidos" por essa linguagem (conforme sintaxe da linguagem).

☞ Dados Literais

São constituídos por caracteres, tais como: **letras, dígitos numéricos e/ou símbolos especiais**.

Dados literais são conhecidos também por:

- **Dados Alfanuméricos,**
- **Cadeia de caracteres, ou**
- **String.**

Nos algoritmos na forma de descrição narrativa, os dados são representados entre aspas (**"**).

Exemplo: **"**Informe código do Cliente**"**,

"Valores em R$**"**

"(10 * 3) / 8 =**"**.

☞ Dados Lógicos

Dados lógicos representam dois únicos valores possíveis:

- **Verdadeiro** ou **Falso**
- **True** ou **False**
- **Sim** ou **Não**
- **0** ou **1** (ou um valor maior que zero)

Esse Tipo de Dado é conhecido também como dado Booleano, uma referência a Boole, que muito contribuiu na área da lógica matemática.

Exemplo: **V -** Valor lógico **Verdadeiro** e

F - Valor lógico **Falso.**

☝ Quadro resumo dos Tipos de Dados

Tipo de Dados			
	Numérico	**Inteiro**	**30 24 0 -12**
		Real	**32.01 0.0 -345. 0. -23.3**
	Literal	**“Informe” “10 + 20 + 30 = ” “ “ “CAFÉ”**	
	Lógico ou Booleano	**V ou F SIM ou NÃO TRUE ou FALSE 1 ou 0**	

☞ Exercícios

Identifique os tipos de dados abaixo, pelas siglas: **NI** (numérico inteiro); **NR** (numérico real); **LI** (literal); **LO** (lógico); **NI-NR** (numérico inteiro ou real):

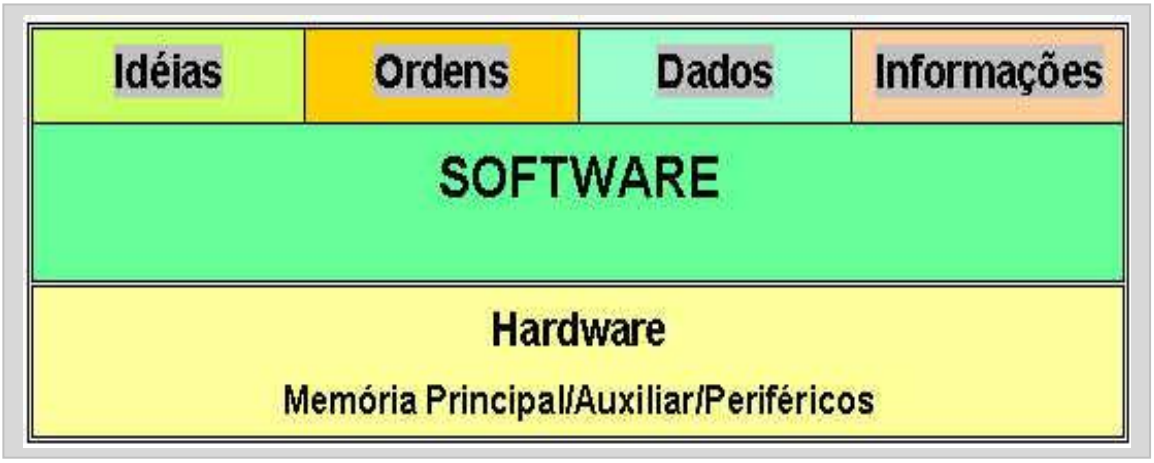
( ) -52	**( ) –0.014**	**( ) –0.0**
( ) +75.	**( ) +0.007**	**( ) "13%"**
( ) 0.0	**( ) +4532**	**( ) V**
( ) F	**( ) "oi"**	**( ) "@terra.com.br"**
( ) –1	**( ) S**	**( ) F**
( ) 0.5	**( ) "+2376."**	**( ) 34**
( ) 1	**( ) "-0.0"**	**( ) "V"**
( ) +44.	**( ) ".F."**	**( ) 0**

3.2 Variáveis – Conceitos e definição

"Na solução algorítmica da maior parte dos problemas, temos a necessidade de guardar valores em determinados passos que serão usados em passos seguintes. De maneira genérica, **variáveis são entidades de um algoritmo que servem para guardar operandos (dados)** que serão usados nos passos seguintes. Ou seja, **elas são a *memória* do algoritmo**." (KOLIVER, 2009, p.33, negrito nosso)

Durante a execução de um programa (algoritmo computacional), o computador está constantemente processando informações, que são **instruções** e/ou **dados**. Para processar dados, o computador armazena-os em posições de memória, denominadas de **variáveis**, pois o **seu conteúdo na memória se modifica durante o ciclo de processamento do programa**.

A figura abaixo mostrar o esquema básico de um *software* (programa ou conjunto de programas).



Analisando essa figura temos que todo *software* é concebido a partir de **idéias oriundas do processo de abstração de situações do mundo real**. As idéias são organizadas logicamente, gerando os **algoritmos** que, conforme já estudado, são conjuntos de instruções (ordens ou comandos) logicamente ordenados, que uma vez executados irá atingir o(s) objetivo(s) propostos pelo algoritmo. Os algoritmos processam dados e informações para produzir os resultados a que se propuseram.

☞ **Como as informações são armazenadas em memória e acessadas pelo computador?**

A memória do computador pode, de maneira simbólica, ser vista como um conjunto de células, onde cada célula (ou um **byte**) pode armazenar um caractere, e possui um endereço, para que o computador possa acessá-la.

Para armazenar dados, precisamos de uma ou mais células agrupadas, dependendo do Tipo de Dados que estamos armazenando, ou seja, um **Dado Lógico irá ocupar 1 célula**, enquanto um **dado alfanumérico, ou mesmo numérico, poderá ocupar n células**.

A definição de variáveis, dentro de um programa, permite ao computador reservar espaços (células) em memória, e atribuir endereços às mesmas. O computador só trabalha com endereços de variáveis, e não com nomes simbólicos.

Após escrever o algoritmo em uma linguagem de programação, gerando um programa (**fonte**), temos que efetuar sua **compilação** (transformação do programa fonte em executável). O compilador, baseando-se nas variáveis definidas (nome, tipo de dado e tamanho), monta uma tabela com nomes simbólicos das variáveis e seus respectivos endereços. Quando em execução, as variáveis são alocadas (reservadas) em endereços de memória RAM do computador onde está sendo executado.

Portanto, ao "carregar" o programa na memória, antes de iniciar a execução dos comandos, o computador aloca posições de memória para as variáveis e constantes definidas no programa, criando uma tabela de variáveis e constantes e atribuindo um endereço para cada uma delas, conforme representado na tabela a seguir:

Endereço	Nome da Variável	Tipo de Dado	Tamanho	Posição Inicial
0FC0	Cod_Cliente	Inteiro	5	0
0FC5	Endereco	Alfanumérico	30	5
0FE8	Cidade	Alfabético	20	35
.....	Sim_Nao	Lógico	1	55

☞ Atributos de uma variável

Cada variável possui basicamente três **atributos**, ou seja:

✓ Atributo 1: **Nome da Variável**

Como em toda linguagem de programação existem **regras para dar nomes à uma variável**. No caso da linguagem C++, as seguintes regras devem ser observadas: o nome da variável,

- **Deve ser único**, em cada bloco de programa.
- **Pode ser de uma letra até palavras com no máximo 32 caracteres**. Caso o nome da variável tenha mais que 32 caracteres, somente os 32 primeiros serão considerados pelo compilador C++.
- **Obrigatoriamente deve começar com uma letra** ou o **caractere especial "_".** Os demais caracteres devem ser **letras, números** ou o **caractere especial "_".**
- **Não deve começar com número.**
- **Não deve conter espaços em branco.**

  Para separar palavras **use o caractere especial "_"** (exemplo: Codigo_Produto), **ou use letras maiúsculas e minúsculas** (exemplo: CodigoDoProduto).
- **Não deve conter caracteres especiais, exceto "_" (*underline*).**
- **A linguagem C é *case sensitive*,** portanto, **letras maiúsculas e minúsculas são consideradas caracteres diferentes** (procurar digitar o programa, sempre que possível, usando letras minúsculas)**.**
- **Não pode ser uma palavra reservada do compilador C++.**

  Exemplos de palavras reservadas da linguagem C++:

auto	double	int	struct
break	else	long	switch
case	ebun	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
asm	pascal	far	huge
interrupt	near	_cs	_ds

.

Importante!!! – Ao nomear uma variável, procurar sempre dar nome significativo, de forma que nos lembre a função que ela terá no contexto do programa e/ou o significado do dado que ela irá conter.

✓ Atributo 2: **Tipo de Dado**

Os dados representam as informações que serão processadas pelo computador, **o tipo de dados atribuído a uma variável define que tipo de informação** essa variável poderá armazenar.

☞ **Tipos de Dados pré-definidos em linguagem C++**

A linguagem de programação C++ possui um conjunto de cinco tipos de dados básicos (pré-definidos ou primitivos)**, à disposição dos programadores**, conforme mostrado no quadro abaixo.

Esses tipos de dados podem ser: **Numéricos** (inteiros, e de ponto flutuante ou com decimais), **Alfanumérico** (caractere), e **Sem Valor** sendo este último utilizado na declaração de função.

Tipos	Descrição	Tamanho Aprox.	Faixa Mínima
char	Caractere	8 bits = **1 byte**	-128 a 127
Int	Inteiro	16 bits = **2 bytes**	-32768 a 32767
float	Ponto flutuante	32 bits = **4 bytes**	7 dígitos de precisão
double	Ponto flutuante de precisão dupla	64 bits = **8 bytes**	10 dígitos de precisão
bool	Lógico	8 bits = **1 byte**	1 ou > = true; 0 = false;
void	Sem valor		

Observações

- ✓ Além dos tipos acima, temos o tipo de dado Lógico**: bool**.
- ✓ **Variáveis do tipo: char** são normalmente utilizadas para conter valores definidos pelo conjunto de caracteres ASCII.
- ✓ A linguagem C++ não possui o tipo de dado: ***string***, porém possui uma biblioteca com diversas funções que manipulam *strings* **que são definidas como cadeia de caracteres do tipo de dado: char.**
- ✓ O tipo de dado**: void,** declara explicitamente uma função que não retorna valor algum ou cria ponteiros genéricos.

☞ **Modificadores de Tipos de Dados básicos em linguagem C++**

Em C++ existem alguns modificadores de faixa (ou intervalo) de dados que uma variável pode armazenar, além de modificar o do dado contido na variável. Portanto, o uso desses modificadores altera a faixa de valores suportada pela variável (mas a capacidade de armazenamento (bytes) continua a mesma). São eles: *short*, *long*, *unsigned* e *signed*.

A tabela a seguir mostra os principais modificadores aplicados aos tipos de dados básicos. Além disso, essa tabela mostra, também, o **formato para leitura do dado** quando se usa o comando de entrada de dados: ***scanf*.**

Tipo	Num de bits	Formato para leitura com scanf	Intervalo	
			Inicio	Fim
Char	8	%c	-128	127
unsigned char	8	%c	0	255
Signed char	8	%c	-128	127
Int	16	%i ou %d	-32.768	32.767
unsigned int	16	%u	0	65.535
signed int	16	%i ou %d	-32.768	32.767
Short int	16	%hi ou %hd	-32.768	32.767
unsigned short int	16	%hu	0	65.535
signed short int	16	%hi ou %hd	-32.768	32.767
Long int	32	%li ou %ld	-2.147.483.648	2.147.483.647
signed long int	32	%li ou %ld	-2.147.483.648	2.147.483.647
unsigned long int	32	%lu	0	4.294.967.295
Float*	32	%f, %e ou %g	3,4E-38	3.4E+38
Double*	64	%lf, %le ou %lg	1,7E-308	1,7E+308
long double*	80	%Lf	3,4E-4932	3,4E+4932

✓ Atributo 3: **Dado contido na variável**

O terceiro atributo de uma variável é o seu conteúdo.

☞ **Declarações de variáveis** em um programa em linguagem C++.

As variáveis em um programa em C++ devem ser declaradas, **geralmente, em bloco de comandos** (delimitado por { }) **antes de serem utilizadas**.

Exemplo:

```
#include<stdio.h>
#include<stdlib.h>
int  main()

{                                                    // INÍCIO do Bloco de Comando

   // Definição de variáveis

   int numero, dobro;

   puts("<< Programa para calcular o dobro de um numero informado >>");

   printf("Digite um Numero: ");              // Solicita um número
   scanf("%d", &numero);                      // Lê número digitado

   dobro = numero * 2;

   printf("\n O dobro de %d é %d \n", numero, dobro);

   return 0;
}                                                    // FIM do Bloco de Comando
```

Entretanto, em linguagem C++ as variáveis e constantes utilizadas no programa **podem ser declaradas em três lugares básicos** no programa, a saber:

1. **Dentro de Funções ou dentro de qualquer Bloco de Comandos na função**.

   As variáveis definidas dessa forma são denominadas **VARIÁVEIS LOCAIS.**

   No exemplo anterior as variáveis **numero** e **dobro** estão definidas dentro da função: **int main()**, que contém o programa principal. Portanto, essas variáveis são LOCAIS.

   Portanto, só podem ser referenciadas por comandos que estão dentro dessa função (main). Observe que o corpo da função principal (*main*) é delimitada por **{ }**. Exemplo:

```
#include <stdio.h>
#include<stdlib.h>

int main()

{                                          // Início do bloco de comandos

    int numero1, numero2;

    numero1 = 10;
    numero2 = 20;

    return 0;
}                                          // Fim do bloco de comandos
```

- **Dentro de um Bloco de Comandos na função**.

  O exemplo a seguir mostra a definição de variável dentro de um bloco de comando fa função: *main*. Embora isso seja possível, dependendo da quantidade variáveis usadas no programa é desejável que as mesmas sejam definidas no início da função.

```
#include <stdio.h>
#include<stdlib.h>

int main()

{                                          // Inicio do bloco de comandos
    int opcao;

    printf("Informe uma Opção:");
    scanf("%d", &opcao);

    if (opcao == 1)
    {
       char s[30];                         // criação da variável

       printf("Entre com o nome");
       gets(s);
    }                                      // destruição da variável

   return 0;
}                                          // Fim do bloco
```

Observação: Variáveis Locais existem apenas enquanto o bloco de comando ({ }) em que foram declaradas está sendo executado. Após execução elas são "destruídas", ou seja, deixam de ocupar espaços em memória.

2. **Na definição dos parâmetros de Funções**.

Dentro de um programa podemos criar várias funções, que serão executadas dentro do corpo do programa principal - **int *main*() { ... }**

As variáveis definidas dessa forma são denominadas **PARÂMETROS FORMAIS**. Exemplo:

```
#include <stdio.h>
#include<stdlib.h>

int func_digito(int cod_cliente)
{
   // calcula digito de controle do código do cliente informado
}

int main()                  // programa principal
{
    int codcli, digito;
    printf("Informe o Código do Cliente:");
    scanf("%d",&codcli);

    digito = func_digito(codcli);

    printf("Digito de controle calculado: %d",digito);

    return 0;
}
```

Analisando o exemplo de programa acima, temos a definição da função: **func_digito**, cuja finalidade é calcular o digito de controle de um código de cliente, contido na variável **codcli** definida e passada no corpo do programa principal (*main*).

A func_digito irá receber esse dado como um **parâmetro formal**: **cod_cliente**, que está declarado no seu cabeçalho.

Observe que a função: func_digito é declarada como uma variável do tipo: **int.** Portanto, após os cálculos, o digito de controle do código de cliente estará disponível dentro do endereço da função: func_digito, na memória do computador.

No corpo principal do programa, o conteúdo da função (variável): **func_digito** está sendo movimentado (=) para dentro da variável **digito**.

3. **Fora das funções**.

As variáveis definidas dessa forma são denominadas **VARIÁVEIS GLOBAIS.**

As variáveis globais são criadas declarando-as fora de qualquer função. Neste caso, essas variáveis são reconhecidas pelo programa inteiro e podem ser usadas por qualquer bloco de código, inclusive de funções. Exemplo:

```
#include <stdio.h>
#include<stdlib.h>

int num1;                  // variável Global

int main()                 // programa principal

{
   int num2, num3;         // variáveis Locais

   num1 = 120;

    if num2 != num3

     {
       num1 = num2 + num3;
       printf("%d",num1);
     }

    return 0;
}
```

Importante! Sobre Variáveis Globais:

- ✓ Encontram-se armazenadas em uma região fixa da memória, separada para esse propósito pelo compilador C.
- ✓ São úteis quando o mesmo dado é usado em muitas funções e/ou blocos de comandos, em um programa.
- ✓ Ocupam memória durante todo o tempo em que o programa estiver executando, portanto, **evitar usar variáveis globais desnecessariamente.**

☞ Atribuição de valor para variáveis, em C++

A atribuição de valores para variáveis, em linguagem C++, é feita **no momento da sua definição,** através de um operador de atribuição (igual) =, ou **nos blocos de comandos** do **corpo principal do programa**, do procedimento, ou da função, através desse operador de atribuição, ou de comandos de leitura de dados (***scanf*** ou ***gets***).

Em linguagem algorítmica o operador de atribuição é representado por uma seta (← **).**

Exemplo de atribuição de valores à variáveis:

```
{
  int mes = 12;                  // Atribuição de valor na definição da variável
  char letra = 'A';

  scanf("%f",&salario);          // Atribuição de valor através de comandos de leitura
  gets(nome_do_cliente);
  {
      indice = 1;   { representação Algorítmica -  indice ← 1; }
      ano = 2005;
      sim_nao = True;  // variável: bool sim_nao;
      nome_do_cliente = 'João Maria José';
      valor_do_item = -250.89;
      valor_icms = ValorTotalNF * 0.12;
  }
}
```

3.3 Constantes – Conceitos e definição

Constantes são expressões **(posições de memória) que têm valores inalterados** durante o ciclo de processamento do programa, sendo seu tipo definido por seu conteúdo.

As declarações de constantes em um programa C++, podem ser feitas:

- ✓ **Dentro de uma função**, usando o modificador **const** antes da definição da constante, ou
- ✓ **Fora de uma função**, usando a **diretiva de compilador**: **#define.**

Exemplo:

```
#include <stdio.h>
#include<stdlib.h>

// Usando diretiva (#) do compilador

#define pi 3.14159265358979323846264338327950    // constante Global

#define ano_atual 2017                           // constante Global

int main()                  // programa principal

{
    int num2, num3;                  // variáveis Locais

    const  int num1 = 120;           // constante Local

    printf("%.4f",pi);

    printf("%d",ano_atual);

    printf("%.d", num1);

    return 0;
}
```

Observações:

- ✓ **Constantes do tipo: char (caractere) são delimitadas por aspas simples (apóstrofo) ['].**

  Exemplo: **const char letra** { **letra = 'a'; letra = '%'; letra = '1';** }

- ✓ **Caracteres delimitados por aspas duplas ["] são tratados como constantes string** (cadeia de caracteres). Portanto, um constante do tipo de dado: char, definida como cadeia de caracteres é considerada uma string.

  Exemplo: **char letras[30];** essa constante irá ocupar de 0 a 29 posições de memória, sendo que o final da cadeia conterá um **'\0'**, que indica o fim da cadeia.

  Exemplo: um caractere delimitado por aspas duplas, ocupará um byte a mais na memória (ex: "a" = 'a' + '\0'), pois é tratado como *string*.

- ✓ **Constantes inteiras são especificadas como números sem componentes fracionários** (ex: 69, -10).
- ✓ **Constantes reais podem serem expressas na forma decimal**, neste caso é requerido o ponto decimal **[.]** seguido da parte fracionária (ex: 3.141, -7.33); **ou através de notação científica** (ex: 4.34e-3). (MARTINS, p.14).

3.4 Strings

Ao contrário de outras linguagens de programação, a Linguagem C (ou C++) não possui o tipo de dado (TD) ***string***. Portanto, **nessa linguagem, para manipular variáveis que contém *strings*** (cadeia de caracteres) **é necessário definir um vetor de caracteres (*char*)** para representá-las, ou seja, para armazenar palavras ou frases (cadeia de caracteres), é necessário que criar uma variável que permita armazenar esses dados, e também estabelecer um limite máximo de caracteres que a variável "*string*" irá armazenar.

Quando se definem algoritmos que utilizam strings, pode haver necessidade de executar alguns procedimentos dependendo da solução do problema que está se tentando resolver através do algoritmo. Dentre esses procedimentos destacam-se:

- ✓ **Comparação de *strings*;**
- ✓ **Copiar uma *string*;**
- ✓ **Concatenar (agrupar) *strings*;**
- ✓ **Determinar o tamanho de uma *string*;**
- ✓ **Comparar *substrings*;**

A linguagem C disponibiliza várias funções prontas para manipulação de *strings*, e que compreendem os procedimentos citados anteriormente.

Exemplo 1

Para um melhor entendimento do uso de *strings*, pela linguagem C++, supõe-se o programa exemplo a seguir, onde serão informadas duas palavras, e em seguida serão mostrados seu conteúdo.

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

main()

{
   setlocale(LC_ALL,"portuguese");

   char palavra1[20], palavra2[20];

   printf("\n\n << Uso do procedimento scanf() para ler strings >>\n");

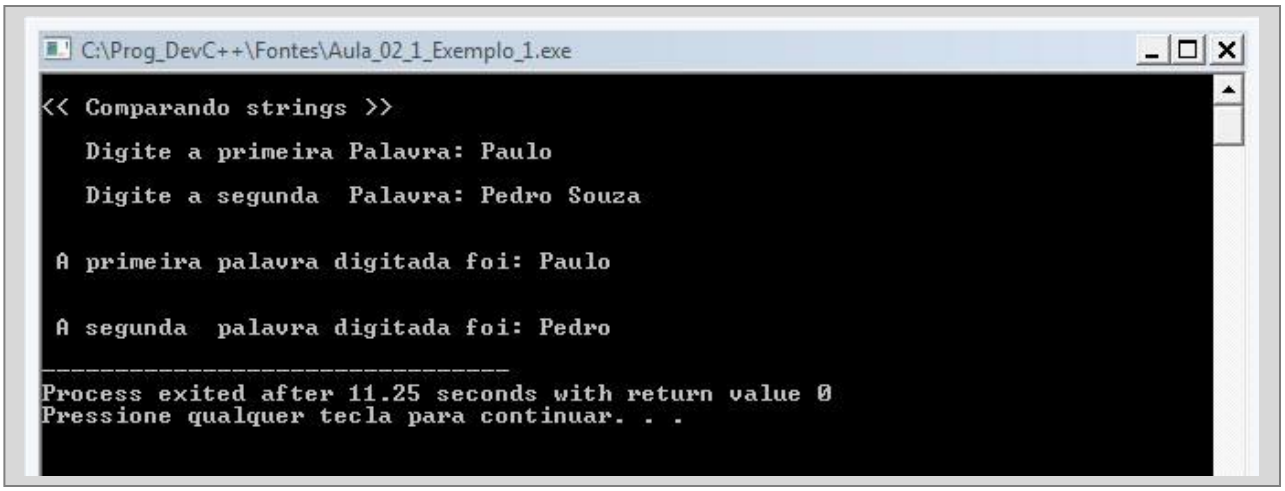
   printf("\n   Digite a primeira Palavra: ");
   fflush(stdin);
   scanf("%s",&palavra1);

   printf("\n   Digite a segunda  Palavra: ");
   fflush(stdin);
   scanf("%s",&palavra2);

   printf("\n\n A primeira   palavra digitada foi: %s\n",palavra1);
   printf("\n\n A segunda  palavra digitada foi: %s\n",palavra2);

   return 0;
}
```

A figura abaixo mostra a tela de execução do código acima.



Podemos observar, nessa execução, que a **primeira palavra** digitada foi: **Paulo**, e a **segunda** foi: **Pedro Souza**. Entretanto, o computador mostrou que armazenou somente a primeira palavra da segunda que foi digitada, ou seja, "**perdeu"** o sobrenome **Souza**, isto é, esse dado **não ficou armazenado** na variável: **palavra2**.

Esse problema ocorreu devido o uso do **comando *scanf***, pois este armazena os caracteres digitados somente até encontrar um espaço em branco.

Para resolver isso deve-se usar a **função *gets*** (*get string*, ou seja, obtenha uma string).

2.4.1 Função *gets()*

Essa função lê uma ***string*** e coloca-a na variável indicada no argumento da função, ou seja, a que está entre parêntesis. Tudo o que for digitado até que seja pressionada a **tecla <ENTER>**, será armazenado na variável, inclusive a digitação da tecla <ENTER> será substituída pelo terminador nulo "**\0**", que será armazenado também na variável.

Todavia, **essa função não é confiável pois, como ela não verifica o tamanho da variável no momento do armazenamento dos caracteres, então poderá ocorrer um overflow (estouro) de memória.** Para contornar esse problema pode-se utilizar a função: **fgets();** onde um de seus parâmetros é o estabelecimento do tamanho do buffer (variável) que irá conter os dados. Exemplo: **fgets(texto1, sizeof(20), stdin);** Ao digitar o conteúdo da variável texto1, se for digitado mais que 19 caracteres, será armazenado somente 19 caracteres mais o **"\0" que significa final da string ou cadeia de caracteres**.

Como já foi observado, uma *string* em C++ será armazenada em um vetor de caracteres (*char*), e sendo um vetor ele deve ter uma tamanho, que é informado entre colchetes no momento da sua definição. Exemplo: ***char* palavra1[20];** portanto, ao se definir um vetor desse tipo deve-se levar em consideração que além da string armazenada, o computador irá armazenar também o "**\0**".

Atenção! Se for ultrapassado o espaço reservado para a *string*, **esta função sobrepõe os valores na memória**, podendo ocasionar um **erro grave**.

A sintaxe dessa função é:

gets(nome_variavel);

Exemplo 2

No programa a seguir, o comando de leitura, ***scanf,* está sendo substituído** pela função ***gets*.**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

main()
{

   setlocale(LC_ALL,"portuguese");

   // Definição de variáveis

   char palavra1[20], palavra2[20];

   printf("\n<< Uso do procedimento gets() para ler strings >>\n");

   printf("\n   Digite a primeira Palavra: ");
   fflush(stdin);
   gets(palavra1);

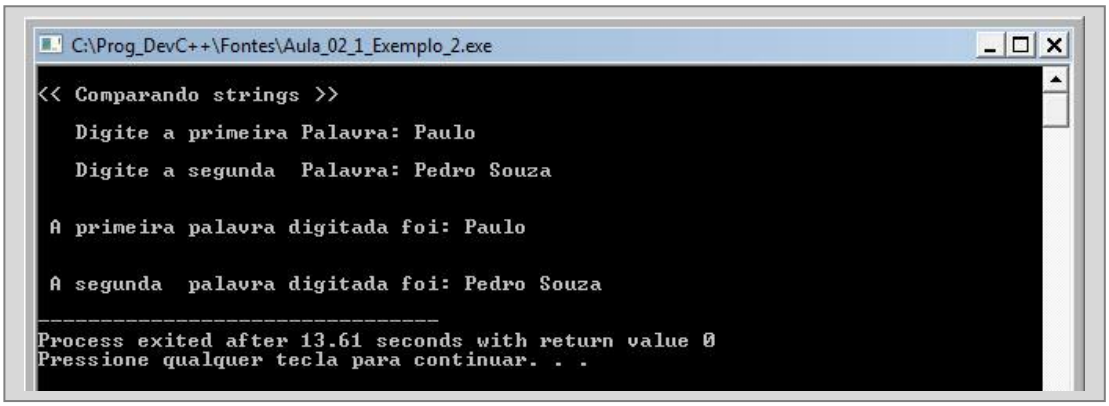
   printf("\n   Digite a segunda  Palavra: ");
   fflush(stdin);
   gets(palavra2);

   printf("\n\n A primeira palavra digitada foi: %s\n",palavra1);
   printf("\n\n A segunda  palavra digitada foi: %s\n",palavra2);

   return 0;
}
```

A função *gets* manipula *strings*, portanto, **é recomendável** incluir a biblioteca **<string.h>.**

Esse programa será executado duas vezes, sendo que na primeira vez observa-se que a segunda palavra digitada não irá "perder" o sobrenome da pessoa.

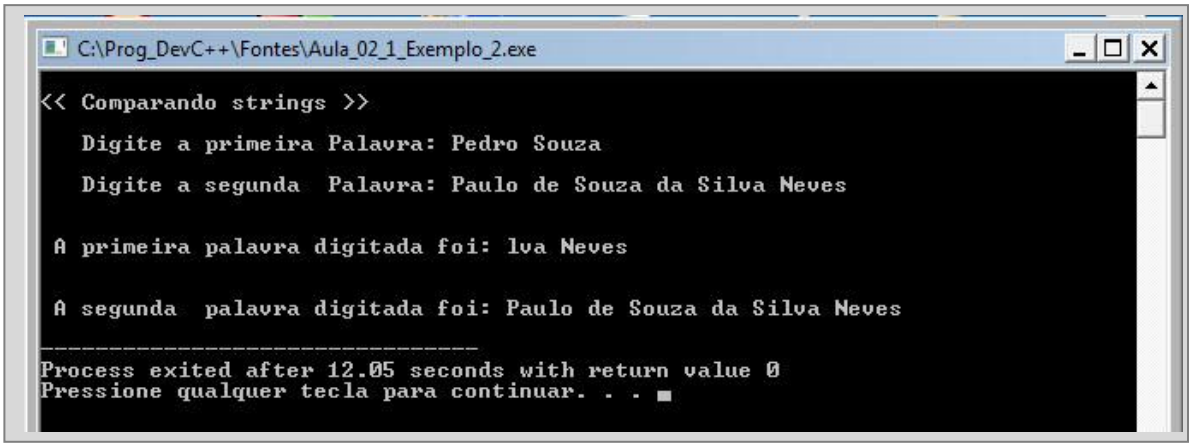


Exemplo 3

Na segunda execução, será digitada a primeira palavra com menos de dezenove letras, enquanto que a segunda palavra irá receber uma palavra com mais de 20 letras.

Observa-se que o resultado foi inesperado, ou seja, quando uma string ultrapassa o limite estabelecido na definição da variável, o computador não "trunca" a variável, ou seja, os dados invadem outras áreas de memória.

Neste exemplo **a área da variável palavra1 foi invadida**.



2.4.2 Função *puts()*

Essa função mostra na tela uma *string* passada como argumento, e em seguida salta para uma nova linha (**\n**). Após sua execução ela irá **retornar um valor não negativo se houver sucesso** no processo de escrita, caso contrário, retorna **EOF**.

☞ A sintaxe dessa função é:

puts(nome_variavel);

Essa função reconhece os mesmos caracteres especiais de escape (aqueles seguidos de ***backslash – "\"***) que a função ***printf()***.

Todavia, a função ***puts()*** requer menos esforço de processamento que a mesma chamada para a função ***printf()***, uma vez que a primeira pode enviar somente strings, enquanto que a segunda, além de strings, pode enviar outros tipos de dados, bem como fazer conversão de formatos.

Portanto, a função ***puts()*** ocupa menos espaço de memória e é executada mais rapidamente que a função ***printf()***.

Exemplo 4

```
    char palavra1[20], palavra2[20];

    printf("\n<< Uso do procedimento puts para exibir strings >>\n");

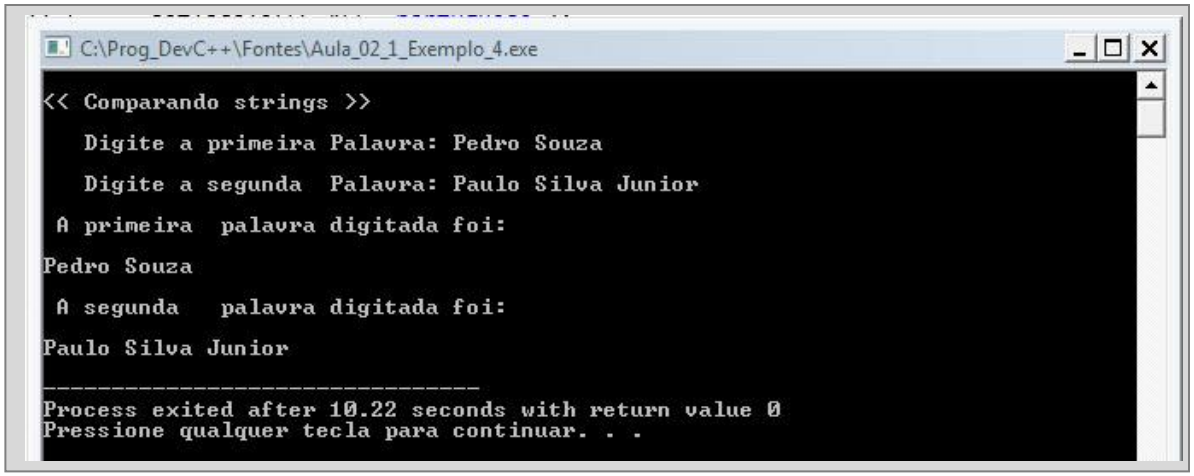
    printf("\n   Digite a primeira Palavra: ");
    fflush(stdin);
    gets(palavra1);

    printf("\n   Digite a segunda  Palavra: ");
    fflush(stdin);
    gets(palavra2);

    puts("\n A primeira  palavra digitada foi: \n");
    puts(palavra1);
    puts("\n A segunda   palavra digitada foi: \n");
    puts(palavra2);

    return 0;
}
```

Exemplo da execução do programa anterior:



2.4.3 Função *strcmp()*- comparação de *strings*

Conforme já estudado anteriormente, em C ou C++, palavras são cadeias de caracteres (**strings**) do tipo *char*, **armazenadas em variáveis compostas**, ou seja, que tem uma dimensão ou tamanho declarado entre **[ ]**, (exemplo: **char palavra1[20];**), e cujo conteúdo é referenciado entre aspas duplas (exemplo: "olá").

Já as variáveis (simples) do tipo *char*, são definidas como caractere único, ou seja, armazenam somente um caractere, que é referenciado entre aspas simples (exemplo: 'E').

Quando queremos **comparar variáveis simples do tipo *char***, usamos **o operador relacional de igualdade (==)**, que poderá retornar uma valor 0 (falso) ou 1 (positivo). Todavia, **para comparar variáveis compostas (strings),** temos que utilizar uma **função**.

☞ A função usada para comparar o conteúdo de uma ***string*** é a ***strcmp()*** que está definida na **biblioteca *string.h*.**

Atenção! Essa função compara o **conteúdo da** variável ***string***, ou seja, se as palavras são iguais, e não se tem o mesmo tamanho.

☞ A sintaxe dessa função é:

strcmp(string1, string2);

Essa função pode **retornar um valor nulo** (zero), **quando forem iguais**, ou **um valor positivo ou negativo**, caso contrário.

Conforme exemplos abaixo, quando as palavras (***strings***) comparadas **são iguais**, a função retorna 0 (verdade).

Quando as palavras comparadas **são diferentes**, **e a primeira é maior**, a função retorna um valor positivo.

Quando as palavras comparadas **são diferentes, e a primeira é menor**, a função retorna um negativo. Exemplo:

strcmp(maior,menor) == 1	*strcmp(**"bb","aa"**) == 1*
strcmp(menor,maior) == -1	*strcmp(**"aa","bb"**) == -1*
strcmp(grande,grande) == 0	*strcmp(**"bb","bb"**) == 0*
strcmp(pequeno,pequeno) == 0	*strcmp(**"aa","aa"**) == 0*

Atenção! No alfabeto a "menor" letra é "a", e a maior, "z".

Exemplo 5

O programa a seguir compara variáveis simples, e compostas, do tipo *char*.

Importante notar que na comparação de variáveis simples, caso a comparação seja verdadeira, será retornado o valor lógico = 1(verdade), caso contrário retornará 0 (falso).

Todavia, **na comparação de *strings* (variáveis compostas)**, caso a comparação seja verdadeira, será retornado o valor lógico = 0(verdade), **pois assim foi construída a função**.

Observar, também, que na primeira execução que **foi informado três caracteres**, porém como a variável é simples, **só foi considerado o primeiro caractere digitado**.

```
 main()
{
   setlocale(LC_ALL,"portuguese");

   bool resultado;

   char letra1, letra2;
   char palavra1[20], palavra2[20];

   printf("\n<< Comparando variáveis simples do tipo char >>\n");

   printf("\n   Digite um    caractere: ");
   fflush(stdin);
   scanf("%c",&letra1);

   printf("\n   Digite outro caractere: ");
   fflush(stdin);
   scanf("%c",&letra2);

   if (letra1 == letra2;)
      printf("\n\n Letra: %c é igual a Letra: %c",letra1,letra2);
   else
      printf("\n\n Letra: %c NÃO é igual a Letra: %c",letra1,letra2);

   printf("\n\n");
   system("pause");
   system("cls");

   printf("\n<< Comparando strings >>\n");

   printf("\n   Digite a primeira Palavra: ");
   fflush(stdin);
   gets(palavra1);

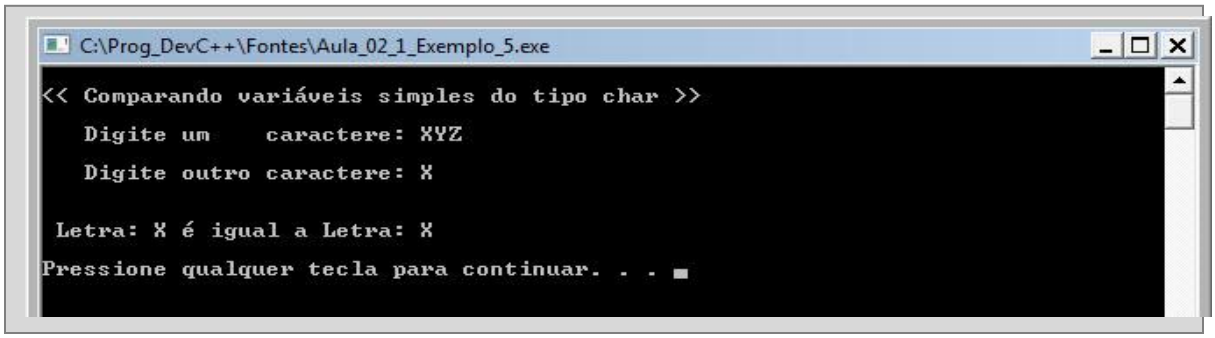
   printf("\n   Digite a segunda  Palavra: ");
   fflush(stdin);
   gets(palavra2);

   resultado =  strcmp(palavra1, palavra2)

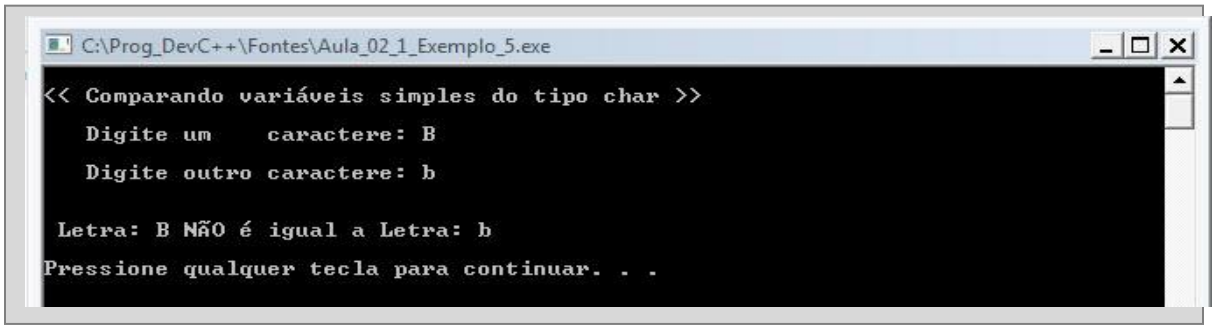
   if ( resultado == 0)
       printf("\n\n A palavra: %s é igual a palavra: %s\n",palavra1,palavra2);
   else
       printf("\n\n A palavra: %s NÃO é igual a palavra: %s\n",palavra1,palavra2);

   return 0;
}
```

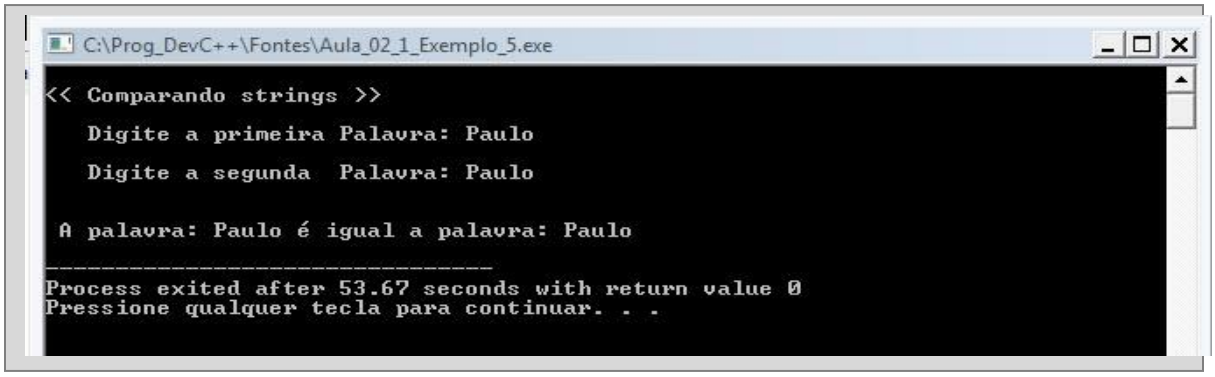
1a. execução



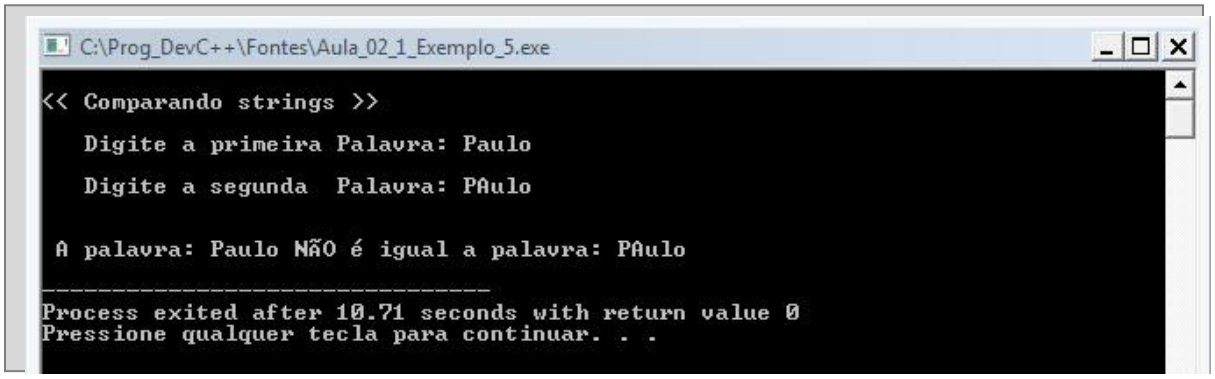
2a. execução



3a. execução



4a. execução



2.4.4 Função *strcpy()* - copiar *strings*

Essa função **copia uma *string* de origem** para **uma *string* de destino**.

A sintaxe dessa função é:

strcpy(string_destino, string_origem);

Exemplo 6

O programa a seguir exemplifica o uso dessa função.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

main()
{
   setlocale(LC_ALL,"portuguese");

   char palavra1[20], palavra2[20];

   printf("\n<< Copiando Strings >>\n");

   printf("\n   Digite a primeira Palavra: ");
   fflush(stdin);
   gets(palavra1);

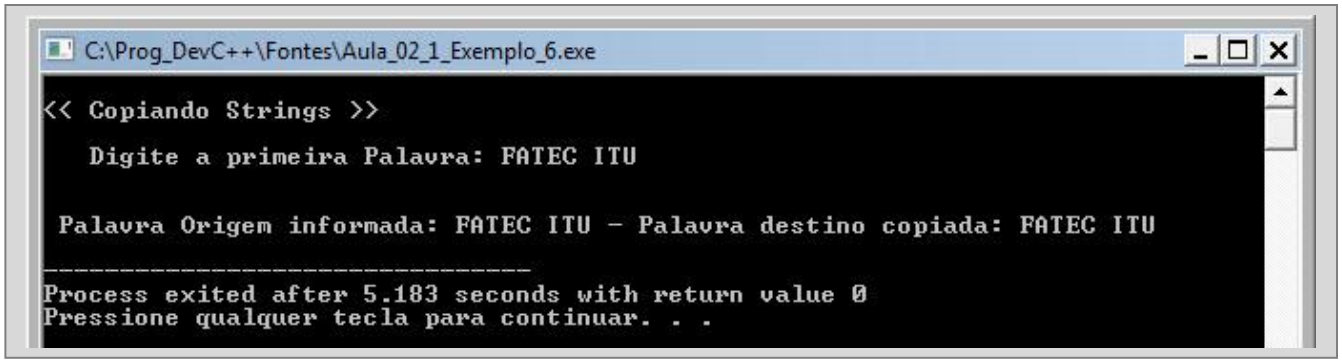
// Copiando strings

   strcpy(palavra2,palavra1);

   printf("\n\n Palavra Origem informada: %s - Palavra destino copiada: %s\n",palavra1,palavra2);

   return 0;
}
```

1a. execução



2.4.5 Função *strcat()* - concatenar *strings*

Essa função **concatena** (agrega) **o conteúdo da *string* de origem** ao final de **uma *string* de destino,** mantendo a *string* de origem inalterada.

A sintaxe dessa função é:

strcat(string_destino, string_origem);

Exemplo 7

O programa a seguir exemplifica o uso dessa função.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

main()
{
   setlocale(LC_ALL,"portuguese");

   char palavra1[20];
   char palavra2[30] = "Linguagem de Programação ";

   printf("\n<< Concatenando Strings >>\n");

   printf("\n   Você está estudando qual linguagem programação? ");
   fflush(stdin);
   gets(palavra1);

// Concatenando strings

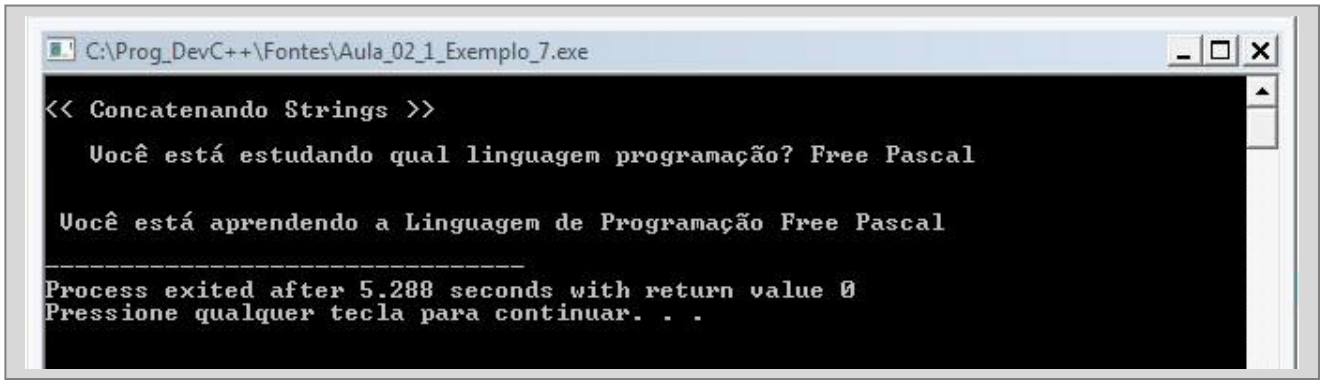
   strcat(palavra2,palavra1);

   printf("\n\n Você está aprendendo a %s\n",palavra2);

   return 0;
}
```

1a. execução

Observar que a *string* digitada (Free Pascal) foi adicionada a uma *string* (Linguagem de Programação) previamente contida dentro da variável palavra2.



2.4.6 Função *strlen()* - determinar o tamanho de uma *strings*

Essa função retorna o tamanho (comprimento) de uma **da *string*** fornecida como parâmetro da função, desconsiderando o terminador nulo (**\0**) da *string*.

A sintaxe dessa função é:

strlen(string);

Exemplo 8

O programa a seguir exemplifica o uso dessa função.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

main()
{
   setlocale(LC_ALL,"portuguese");

   char palavra1[20];
   char palavra2[40] = "Faculdade de Tecnologia de Itu";

   printf("\n<< Determinando o tamanho de strings >>\n");

   printf("\n   Você está matriculado em qual semestre? ");
   fflush(stdin);
   gets(palavra1);

// Determinando o tamanho de uma string

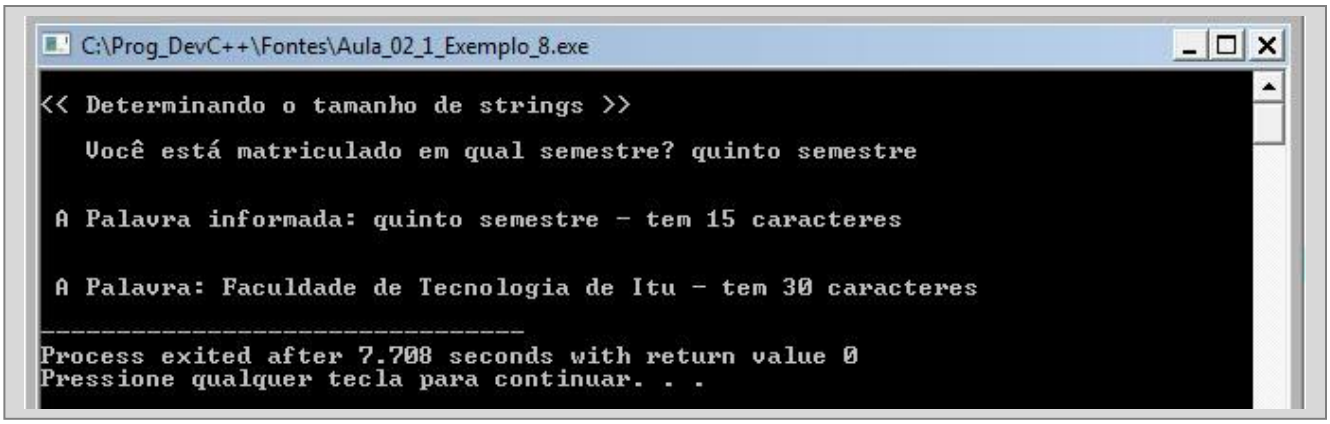
   printf("\n\n A Palavra informada: %s - tem %d caracteres\n",palavra1,strlen(palavra1));

   printf("\n\n A Palavra: %s - tem %d caracteres\n",palavra2,strlen(palavra2));

   return 0;
}
```

1a. execução

Observar que os espaços entre as palavras (espaços em branco), também são considerados caracteres da *string.*



2.4.7 Função *strstr()* - comparar *substrings*

Essa função permite verificar **se uma *substring* está contida dentro de uma *string***, ou seja, verificar se uma *string* está contida dentro outra.

Como exemplo de uso de função seria realizar uma pesquisa de todas as pessoas que começam com um determinado nome ou sobrenome, ou pesquisar um produto que contenha uma determinada palavra, etc., entre outras aplicações.

A sintaxe dessa função é:

strlen(string,substring);

Exemplo 8

O programa a seguir exemplifica o uso dessa função.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

main()
{
   char frase1[40], palavra1[20];

   printf("\n<< Comparando se uma substring está contida em uma string >>\n");

   printf("\n   Digite uma frase: ");
   fflush(stdin);
   gets(frase1);

   printf("\n   Digite uma palavra a ser pesquisada dentro da frase: ");
   fflush(stdin);
   gets(palavra1);

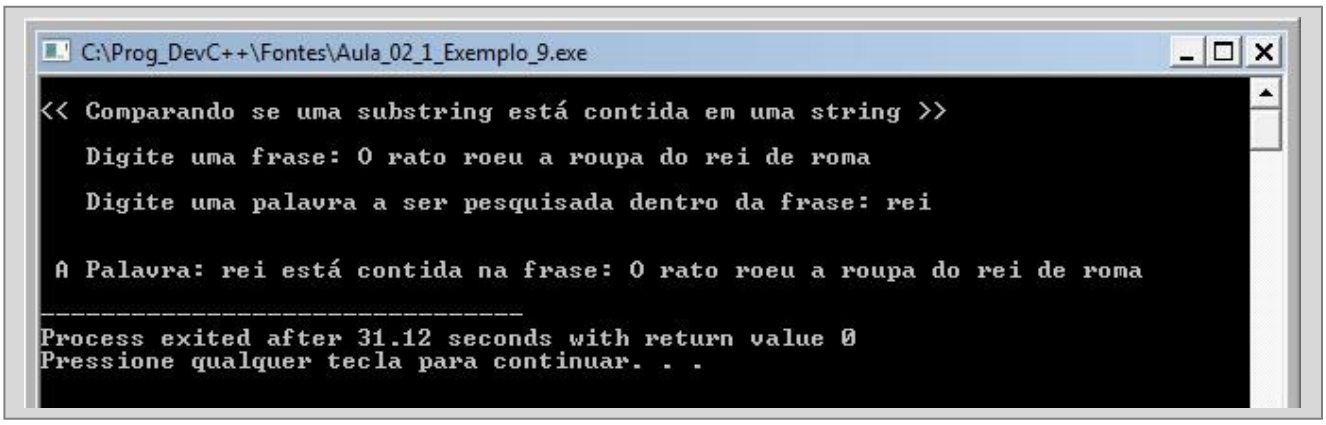
// Comparando substring dentro de uma string

   if (strstr(frase1, palavra1) > 0)
      printf("\n\n A Palavra: %s está contida na frase: %s\n",palavra1,frase1);
   else
      printf("\n\n A Palavra: %s NÃO está contida na frase %s\n",palavra1,frase1);

   return 0;
}
```

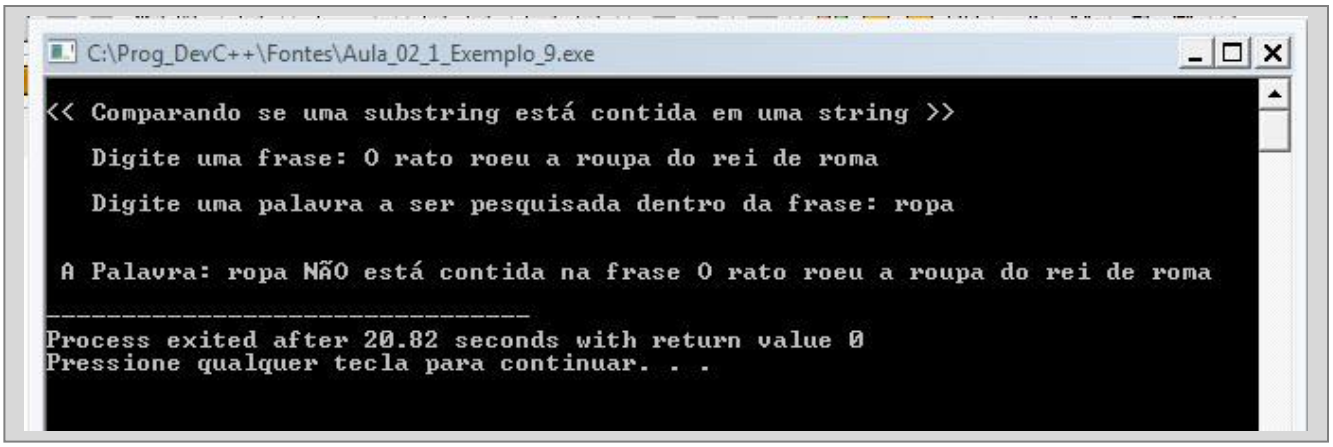
1a. execução

A palavra ("rei") está contida na frase.



2a. execução

A palavra ("ropa") NÃO está contida na frase.



Exercícios:

1. Elabore um algoritmo para calcular e mostrar a área total e o valor de um terreno retangular, tendo como dados de entrada: a Largura, o comprimento e o valor do m^2 do terreno.
2. Elabore um algoritmo para calcular e mostrar o IMC (Índice de Massa Corpórea) de uma pessoa, tendo como dados de entrada: a Altura e o Peso Atual da pessoa.

   A fórmula para cálculo do IMC, é: IMC = Peso Atual ÷ (Altura x Altura)

Bibliografia:

- PIVA JR, D.; ENGELBRECHT, A. M.; NAKAMITI, G. S.; BIANCHI, F.. **Algoritmos e Programação de Computadores**. Ed. Campus-Elsevier, 2012.
- SALVETTI, D. D.; BARBOSA, L. M.. **Algoritmos**.Ed. Makron Books, 1998.
- KOLIVER, C. *et al.* **Introdução à construção de Algoritmos: notas de aulas.** Caxias do Sul, RS: EDUCS, 2009.
- MARTINS, Luiz Gustavo Almeida. **Apostila de Linguagem C (Conceitos Básicos).** Universidade Federal de Uberlândia. Faculdade de Computação.
  Disponível em :http://www.facom.ufu.br/~gustavo/ED1/Apostila_Linguagem_C.pdf